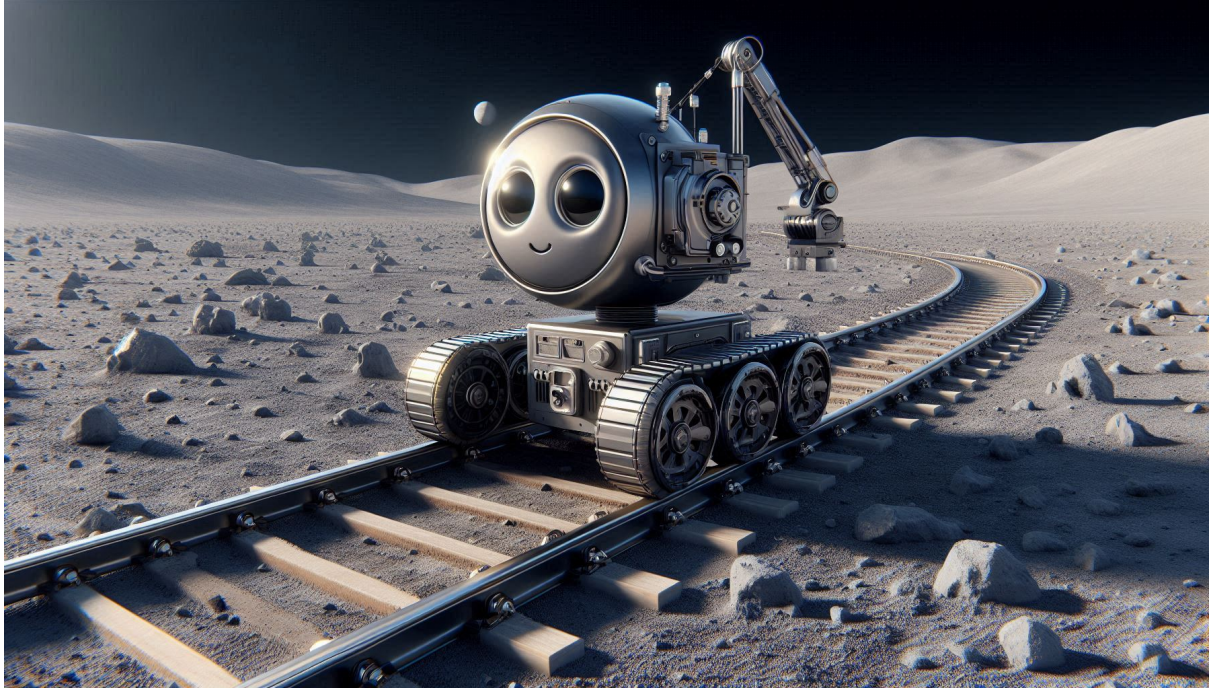


LUNAR RESOURCE SORTING FACILITY



0 Introduction

Dear students

Welcome to the Lunar Resource Sorting Exercise! In this exercise, you will work on a space mission design challenge that introduces you to the core concepts and processes of System Engineering that will follow you throughout your semester — and beyond.

This exercise is a fun competition and is intentionally ungraded. It's here for you to get to know the theoretical concepts and take the time to understand them thoroughly, as you will have to apply them in the graded projects later this semester.

So take your time, dig into the material, and most importantly — have fun!

1	Table of Contents	
0	Introduction	2
2	Background	4
3	Task Description	4
3.1	Hardware Description	4
3.2	Grading & Prize	5
4	Timeline.....	6
5	Deliverable: Data Package 1&2	7
6	Deliverable: Design Review.....	7
7	System Demonstration.....	8
7.1	Setup and Rules.....	8
7.2	Boundary Conditions	8
7.3	Scoring.....	9
7.3.1	How you gain points.....	9
7.3.2	How you lose points	10
8	Deliverable Components and Glossary	10
8.1	Data Package Components	10
8.1.1	Mission Statement.....	10
8.1.2	Principal Players.....	10
8.1.3	Product Tree.....	11
8.1.4	Requirements Flowdown	11
8.2	Design Review Components.....	12
8.2.1	Concept of Operations	12
8.2.2	System Overview	12
8.2.3	V-Model and FMEA	12
8.2.4	Demonstration (Voluntary).....	13
9	Scoring Tables	14
10	Coding Help.....	16
10.1	Installing the VEX VS Code Extension	16
	Using VEX VS Code	19
10.2	Important Hints & Links.....	21
10.2.1	Documentation	21
10.2.2	Numpy, Matplotlib, etc.	21
10.2.3	Getting Data to the Computer	21
10.2.4	General Hints	21
10.2.5	Manual Control	22
10.3	Testing Sensors and Motors.....	23

2 Background

An exploratory mission to the Moon has discovered a region abundant in resources, containing three distinct types of raw materials. In response, a space agency plans to build a lunar resource extraction and sorting facility to harvest them.

The mission's success relies on a robotic system, mounted on rails, capable of collecting materials from the Mining Area, transporting them to a Production Facility, and sorting them into their respective piles.

To find the best design, the agency has launched a public competition. Each participant belongs to a design team competing to win the contract. Your team's challenge is to develop an autonomous system that can collect raw materials and organize them into sorted piles within the Production Facility.

While the general shape and location of the Mining Area have been mapped, the exact spots where each material can be found remain unknown. The layout of the Production Facility, and its position within the Mining Area, has already been fixed by the agency. The earlier exploratory mission left behind a set of rails and a trolley on the lunar surface — your system must build upon this existing infrastructure rather than replace it. This early stage in the Moon base's development offers your team a unique opportunity to integrate your design seamlessly with what's already in place.

To allow testing before deployment, the agency provides teams with a miniature scale model of the Mining Area and Production Facility, built from a VEX kit, on which teams can prototype and validate their robotic system before final deployment.

3 Task Description

Your task is to develop a robotic system and passing the space agency's review process on the way – this includes delivering Data Packages on time, revising them, passing a design review, and ultimately demonstrating your systems capabilities in a live demonstration.

Participants are organized into groups of six, with each group provided one *VEX IQ Education Kit* plus one *VEX Linear Motion Pack*. Each team must build an autonomous robot to identify, collect, transport and sort the resources. Modifying and/or extending the available parts, e.g. using 3D printing, is **not** permitted.

The performance of the system will be assessed in form of a live demonstration, with the goal of collecting the largest number of resources possible in a limited amount of time. A different number of points will be rewarded for different resource types and achieving certain milestones yield additional points. The full rules for the live demonstration can be found in **Section 7**. Make sure to read the rules carefully!

3.1 Hardware Description

The live demonstration will be carried out on a designated playing field (see Figure 1) with the following infrastructure in place:

The "**Mining Area**" (**A**) is a designated space, randomly scattered with "**Resources**" (differently coloured cubes). The Cubes are randomly distributed in the Mining Area. The distribution setup is unknown but equal for all teams.

The "**Production Facility**" (B) consists of a small field with two segmented areas designed for sorting. Here, the Game Objects should be sorted by colour. The Production Facility is only restricted by walls on the side, but not in the back.

The "**Set of Rails**" includes two smooth tracks and one geared track that traverse the mining area. The "**Trolley**" is a motorized structure that rides along these rails. It comes with a fixed interface (see *Interface Control Document*) that cannot be altered, which any system built on top of it must adapt to.

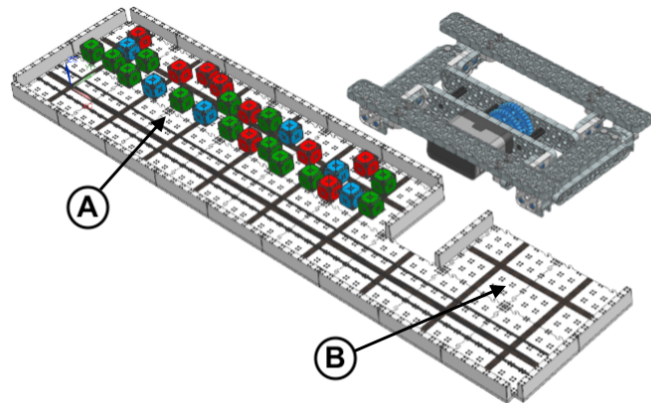


Figure 1: Playing Field, Production Facility and Rail Trolley (not to scale)

You will be able to test your systems beforehand on designated test fields. Keep in mind that the final playing field will be significantly larger and will have much more cubes.

3.2 Grading & Prize

In order to win the competition, your system should not only perform well during the Demonstration, but your team must also deliver good work on the Data Package 1 and 2 and pass the Design Review well. Each of these four assessments will be reviewed with grades from 1- 6 with the following weighting:

	Weighting
Data Package 1	20 %
Data Package 2	40 %
Design Review	10 %
Demonstration	30 %

Scoring guidelines are provided in Section 9 and are accessible in "Scoring.xlsx".

The team with the best overall grade wins the competition. There will be prizes to win!



4 Timeline

The below timeline provides a general outline of all tasks that must be completed. Given the tight schedule, it is recommended that teams divide the subsystems and parts of the Data Packages among members to ensure efficiency.

Time	Monday, 28.09 (1300-1700)		Wednesday, 30.09. (1300-1700)	Monday, 05.10. (1300-1830)
Task 1	Mission Engineering - Mission Requirements - Stakeholder Analysis		Design Review - Presentation of System Design & ConOps - Live Demo of the system (voluntary)	System Demonstration (14:00 – 16:00)
Task 2	Systems Engineering - List of Requirements - Product Tree		Systems Engineering - Subsystem Specification - Final Flow-Down	Wrap-up - Clean-Up - Lessons Learned & Feedback
Task 3	Baseline Design	- ConOps - V-Model/FMEA - Key Requirements	Design Iteration & Testing	Announcement of Winners & Celebration
Self-Study	Design Improvements		Design Improvements	-
Deliverables	Data Package 1 (by Mon 28.09. 22:59)		Design Review (by Wed 30.09. 13:59) & Data Package 2 (by Thu 01.10. 22:59)	System Demonstration

5 Deliverable: Data Package 1 & 2

The Data Package (DP) outlines the functions and constraints necessary for a system to achieve its mission objectives. You have to submit the DP1 on Monday, 28.09. and the revised DP2 on Thursday, 01.10. on Moodle. Both DPs are comprehensive and include the following components:

1. Mission Statement
2. Principal Players (Stakeholders)
3. Product Tree
4. L0 / L1 / L2 Requirements (focus on Key Requirements)

The detailed description of each component is given in Section 8.1.

After the submission of DP1, we will provide feedback to your team according to the scoring table in Section 9. You are expected to revise your package based on this feedback and the insights gained from the Design Review. Incorporating the feedback from DP1 effectively is a critical component of your DP2 score. Note that the scoring weights differ for DP1 and DP2 (the specific weights for DP2 are noted in brackets).

How to submit: Use the file `Data_Package_Template.xlsx` provided, rename it to `Data_Package_Team_<your team number>.xlsx` and submit it on Moodle until **Monday, 28.09.2026 22:59**.

6 Deliverable: Design Review

The Design Review (DR) consists of a 15-minute presentation paired with a voluntary live demonstration of your system's current state. This review serves as a platform to present the baseline design of your system, even if it's still a work in progress. It is crucial to address the following key areas during your presentation:

- Concept of Operations (ConOps)
- System Overview
- V-Model and FMEA
- Live Demonstration (voluntary)

The detailed description of each component is given in Section 8.2.

The Design Review offers a valuable opportunity to receive feedback on the design, implementation, and strategic direction of your project. The more comprehensive your presentation, the more extensive feedback you can expect to gather. The scoring table for the DR can be found in Section 9.

How to submit: Submit the pdf of your slide deck for the Design Review presentation on Moodle until **Wednesday, 30.09.2026 13:59**.

7 System Demonstration

On Day 3 of the Lunar Resource Sorting Exercise, all teams present their design in a final system demonstration which contributes to the final score and hence the overall competition. All teams have to stop working on their system once the first team starts with their demonstration.

7.1 Setup and Rules

The team is responsible for setting up the system in its initial configuration. A five-minute setup period is provided. Once this period ends, the demonstration rules are as follows:

- The system must operate autonomously for 10 minutes, during which it can accumulate points.
- Any interference with the system, such as touching or remote steering, is strictly prohibited and may result in point deductions or disqualification.
- In the event of severe malfunctions, the referee may intervene at the team's request to address the issue; however, the demonstration timer will continue running during this intervention.
- The referee is prohibited from altering the demonstration's boundary conditions once it has begun.
- Teams may attempt an unlimited number of restarts and resets during their allocated time; however, all previously accumulated points will be reset to zero. During a reset the already collected cubes will be returned to the mining area and all cubes will be realigned. The demonstration clock will continue to run and will not be reset. Please note that points will be deducted for each reset performed during the demonstration (see 7.3.2).
- If a cube is dropped during the transportation and no reset is initiated, this cube will be removed from the mining area. Please note that points will be deducted in this case (see 7.3.2).
- The teams are not allowed to modify the hard- or software during the demonstration except during their allocated demonstration timeslot. During their timeslot the teams can do whatever they like, including changing software.

After each run, the referee will place the cubes in a defined but to the team unknown pattern on the playing field. The cubes' orientation will be consistent, and they will be predominantly positioned in a "front row" at a specified distance from the rails, with green blocks mostly populating this area. Red blocks will be predominantly placed behind green or blue blocks, in a collection of cubes behind the first row.

7.2 Boundary Conditions

The placement of the cubes follows the following constraints in minimal distances between one cube and:

- walls: 2 ± 0.5 cm (*parallel to rail*)
- other cubes: 2 ± 0.5 cm (*parallel to rail*)
- the trolley: 4 ± 0.5 cm

The relative rotation of the cubes with respect to the rails is in the range of $\pm 10^\circ$.

The cubes need to be sorted as follows:

- green cubes: piles must be placed in the space of the production facility closest to the Mining Area
- red cubes: must be placed in the space of the production facility further away from the Mining Area

- the two sorted piles of cubes must be separable by a straight line
- any cube whose center of gravity is not inside the production facility will not count as a collected cube.

7.3 Scoring

In the System Demonstration, there are multiple ways to score points. The provided scoring Excel sheet might also help to understand the scoring system better.

7.3.1 How you gain points

- **Placing the cube in the production facility**
 - Placing the cube anywhere inside the production facility generates points
 - **Reward:** 20 points per cube (only red and green)
- **Placing cubes in the production facility separated by colour**
 - Correctly placed cubes generate significantly more points
 - **Reward:**
 - 0 points per blue cube
 - 50 points per green cube
 - 150 points per red cube
- **Achievements**
 - Achievements can be claimed by all teams
 - They can only be achieved once per team
 - They are achieved once the system achieves a certain milestone
 - **Rewards:**
 - “🤖”, the system does not work: 10 points
 - “It Moves!”, the system can move along the rail: 20 points
 - “Comfy Ride”, the system does not drop a cube: 25 points
 - “Tandem”, The system transports two or more cubes at once: 100 points
 - “Half-Hauler”, The system successfully collected 50% of the cubes: 50 points
 - “Treasure Tracker”, The system successfully collected 75% of the cubes: 75 points
 - “Cube Crusader”, The system successfully collected 100% of the cubes: 100 points
 - “Colour Spotter”, The system correctly sensed the colour of 50% of the collected cubes: 50 points.
 - “Spectrum Specialist”, The system correctly sensed the colour of 75% of the collected cubes: 75 points.
 - “Chromatic Champion”, The system correctly sensed the colour of 100% of the collected cubes: 100 points.
- **Decomissioning**
 - The team gets a bonus of 400 points after the demonstration if the system is properly disassembled and the kits are clean and complete

7.3.2 How you lose points

Points can also be deducted. The following events cause a point deduction:

- Losing a cube during transportation (unintentional drop of a cube)
 - **Deduction:** 25 points for green and blue cubes
75 points for red cubes
- Resetting the mining area
 - **Deduction:** 100 points per reset
 - **Note:** *This is the only deduction which is not reset during a restart!*
- The team manually influences the autonomously operating system (except for reset procedure)
 - **Deduction:** *At referee's discretion. Ranges from 100 points to disqualification.*
- It is possible for teams to demand the removal of a single cube from the playing field by the TA's
 - **Deduction:** 250 points per cube removed

The final grade for the demonstration will be calculated by linearly mapping the percentage of points you achieve (out of the maximum possible) onto a scale from 1 to 6.

The competition is built on fair play! If you discover any loopholes in the rules, please report them to the TA team. The TA team may allow certain strategies, if they are deemed within the spirit of competition. Attempting to exploit loopholes or undermine the intent of the rules without approval of the TA team may result in disqualification.

For example, submitting an empty ConOps and not building a robot might earn you the achievements “🤖”, “Comfy Ride”, and “Chromatic Champion” totalling 135 points. However, this will also earn you a disqualification, as such actions go against the spirit of the competition.

8 Deliverable Components and Glossary

8.1 Data Package Components

8.1.1 Mission Statement

The Mission Statement articulates the qualitative goal of a system. It doesn't include numerical values but provides a broad direction for the project's trajectory.

8.1.2 Principal Players

This is a simplified Stakeholder Analysis, based on SMAD (Space Mission Analysis and Design, by J.R. Wertz). It is the sole deliverable that should be completed in a “story frame”, which means it should be framed within the narrative of your mission. For example, the project is not initiated by the Master Programme, but by a major space player such as the European Space Agency.

Table 1: The principal players and their objectives (taken from the SMAD, Table 3-2)

Mission	FireSat II	SCS	Safari Airplane
Primary Customer / Objective	US Forest Service / Detect and monitor forest fires in US and Canada using as little financial resources as possible	Both DoD and Disaster Relief Agency / Supplemental, continuous, real-time communications	African safari operator / Improve profits by moving people quickly between sites
Secondary Customer / Objective	Congress / Demonstrate to the public that action is being taken and government funds spent wisely	Congress / Satisfy military needs and, at the same time, respond to public perception of slow disaster relief support	National Bank of Kenya / Make money on a commercial loan; support increased safari business
Operator / Objective	FireSat II Mission Operations Office / Provide forest fire data to the end user in real time; archive data; monitor and maintain the health and safety of FireSat II	SCS Spacecraft Operations Office / Monitor and maintain the continuously operating communications link; monitor and maintain the health and safety of the satellites	2 pilots hired by the safari company / Provide rapid, safe interlocation transportation
End User / Objective	Forest fire field management or firefighter in truck / Track current location of fire relative to themselves	Individual soldier or relief worker in the field / Explain situation and get the right help where it's needed as quickly as possible	People on safari / Don't care about airplane—just want to see as much as they can in 10 days
What Might Change System Objectives	Change in administration; change in projected system cost (up or down); major fire that didn't have sufficient monitoring resources	Splitting procurement to buy 2 systems; major disaster where communications weren't available in a timely manner	Safari company sold to either much bigger or much smaller company; a new, very remote site for wildlife viewing is opened

8.1.3 Product Tree

The Product Tree provides a hierarchical breakdown of the system, where each "root" of the tree fulfills a specific function. The Product Tree divides the system into subsystems and not subdomains. Additionally, it's important to note which Level 1 (L1) requirement each subsystem addresses.

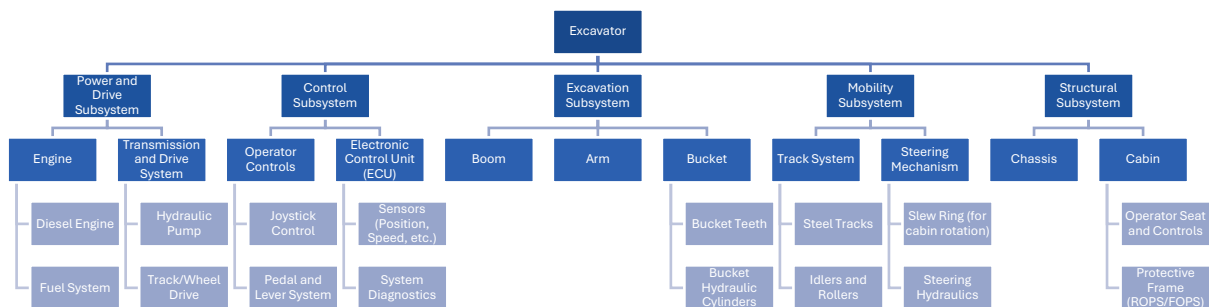


Figure 2: Product tree of an excavator

8.1.4 Requirements Flowdown

For this exercise, the requirements need to be organized into three levels:

- **L0 (Mission-Level) Requirements:** These are broad and conceptual, addressing the mission's overall purpose, success criteria, and high-level constraints.
- **L1 (System-Level) Requirements:** L1 requirements detail the specific tasks and actions required of the entire system to fulfill the L0 requirements. Subsystems are delineated based on these requirements.
- **L2 (Subsystem-Level) Requirements:** These provide detailed and technical descriptions of how specific subsystems operate and interact to meet the L1 requirements.

It is essential to maintain a clear traceability from lower to higher levels of requirements. Each requirement must be *quantifiable or have specific pass/fail criteria* that allow for system testing. This structured list of requirements is crucial for the verification process. Make sure to highlight and describe the *key requirements*.

8.2 Design Review Components

8.2.1 Concept of Operations

The Concept of Operations (ConOps) outlines the sequence of actions or operational steps a system undergoes to meet mission requirements. It provides a broad overview of system functionality from the user's perspective, detailing the specific operations, their timing, and the context in which they are performed. ConOps is less about individual functionalities and more about describing how various components of the system interact over time to achieve the overall mission objectives. This approach helps stakeholders understand the system's operational flow and decision-making processes. For your presentation, try to illustrate your ConOps in an intuitive way!

8.2.2 System Overview

Based on the Product Tree, present the architecture of your system. Ensure that each subsystem is accurately depicted to enhance understanding of the system's architecture. The interfaces between subsystems, and how the subsystems interact to form a coherent system, is a core element of the System Overview. Each subsystem fulfils a function necessary to complete the mission as outlined in the ConOps. You can expect your audience to have read your DP1 for the presentation.

8.2.3 V-Model and FMEA

Analyze your system in terms of the V-model (see figure below) and conduct an FMEA (Failure Modes and Effects Analysis) of a **subsystem** of your choosing according to the following tasks:

Task 1 (left-hand side of the V-model): create a break-down of your system into its subsystems and modules starting from the product tree and L0 requirements you defined in the DP. Note that it is helpful to work on this task in parallel with the requirements breakdown of DP1.

Task 2: Pick an arbitrary subsystem and create a first design/prototype according to its requirements.

Task 3 (right-hand side of the V-model): once you know the design of the module in task 2, conduct an FMEA to determine its potential failure modes and the associated effects on the system, and answer the following questions:

- a. Are there any single points of failure in your design?
- b. Which failure mode is the most critical (in terms of probability of occurrence and severity)?
- c. How could you test the module (or the integrated system) with respect to that failure mode?
- d. What measures can you take to mitigate the risk of that failure mode?

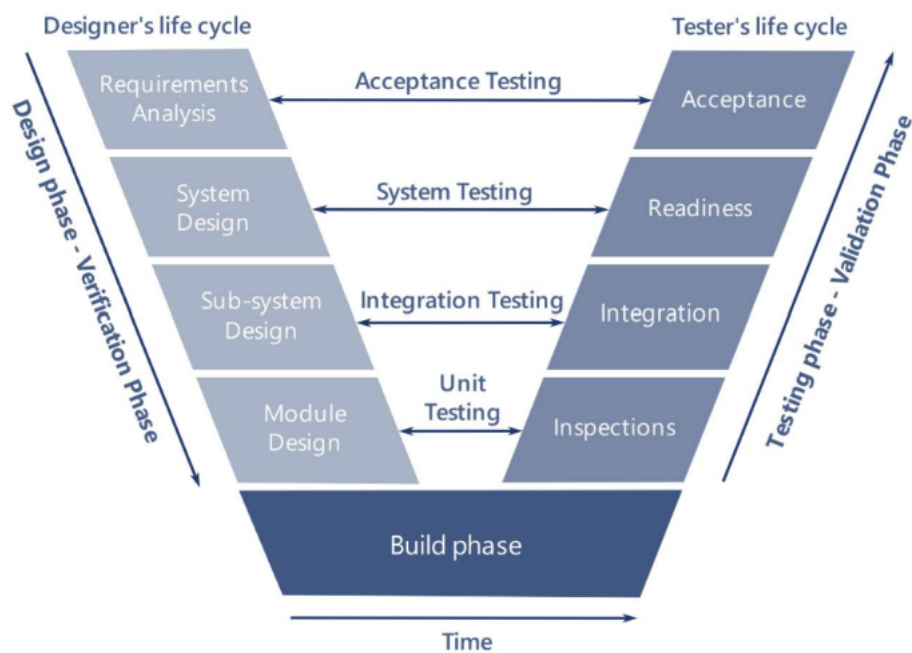


Figure 3: V-Model

We suggest the following structure for the presentation of the V-Model and FMEA. Feel free to follow it or come up with your own structure, as long as it covers the same content:

- Show the L0 requirements and walk the audience through the design phase according to the left-hand side of the V-model (Task 1).
- Provide a drawing or VEX visualization of the chosen subsystem and give a short explanation why you chose the specific design (Task 2).
- FMEA matrix or similar, elaborate on which is the most critical failure mode and answer the 4 questions of Task 3.

8.2.4 Demonstration (Voluntary)

Present the baseline design of your system during a live demonstration. The goal is to showcase the system's ability to autonomously execute the sequence of actions described in the Concept of Operations. At this stage, full autonomy is not required, but demonstrating as much autonomy as possible will help illustrate the system's functional flow and allow for more detailed feedback. If the system is only partially autonomous, provide a roadmap for achieving full autonomy.

9 Scoring Tables

Table 2: Data package scoring

Task	Excellent (5.5 – 6)	Good (5.0 – 5.5)	Fair (4.0 – 5.0)	Lacking (1 – 4.0)
Mission Statement 15 % -> DP1 (5 %) -> DP2	Statement fully aligns with mission, uses correct structure, 0–1 minor language error, ≤ 1 deviation from form	Minor ambiguity or ≤ 2 form deviations	Lacks specificity, ≥ 3 form deviations	Vague, irrelevant, or ignores prescribed form
Principal Players 10 % (5 %)	All players named, each role relevant & clear	≥ 75% players clear & relevant	≥ 50% players clear & relevant	Less than half identified or relevant
Product Tree 25 % (15 %)	All subsystems listed, logical hierarchy, no major omissions	≤ 1 missing subsystem or minor hierarchy gap	≤ 3 missing or irrelevant elements, wrong structure	Structure unclear or wrong, ≥ 4 major omissions
Requirements: Clear and Measurable 30 % (20 %)	≥ 90% requirements have measurable values, no ambiguity, 1 per statement	≥ 80% measurable & clear, 1 per statement	≥ 60% measurable & clear, multiple requirements per statement	< 60% measurable, multiple ambiguities
Requirements: Traceable 20 % (15 %)	100% correctly traceable L2→L1→L0	≥ 90% traceable	≥ 70% traceable	< 70% traceable
Implementation of Feedback and consistency within deliverables 0 % (25 %)	All feedback addressed, justification for all changes is given. 100% consistent content through all deliverables	≥ 80% addressed Minor inconsistencies	≥ 60% addressed, not all changes are justified. Major differences between different deliverables	< 60% addressed Completely inconsistent
Key Requirements 0 % (15 %)	All key requirements identified & justified. Completely aligned to system goals.	≤ 1 missing key requirement. Strong justification.	At least 1 key requirement identified, partial justification.	No key requirements identified.

Table 3: Design review scoring

Criteria	Excellent (5.6 - 6)	Good (5.0 – 5.5)	Fair (4.0 – 4.9)	Lacking (1 – 3.9)
ConOps 20 %	All operational phases identified, clear sequencing, no major logical gaps, fully aligned with mission	≥ 80% of phases clear and sequenced, ≤ 1 phase misaligned with mission, only minor aspects not considered	≥ 50% of phases clear and sequenced, ≤ 2 phases misaligned with mission, ≤ 1 major aspect not considered	< 50% of phases clear, weak mission alignment, main issues not addressed, misleading visuals
System Overview 15 %	All subsystems shown with correct interactions and interfaces, supported by clear diagrams	≥ 80% subsystems and interfaces described; minor clarity gaps	≥ 50% subsystems/interfaces covered; ≤ 2 major clarity gaps	Architecture incomplete or unclear; interactions missing or incorrect
V-Model 10 %	System break-down in accordance with V-model and L0 req., all subsystems included	System break-down ≥ 80% in accordance with V-model and the L0 req., max. 1 subsystem missing	Break-down is ≥ 50% in accordance with the V-model and the L0 req., max. 2 subsystems missing	Break-down is < 50% in accordance with the V-model and the L0 req., more than 2 subsystems missing
FMEA 10%	FMEA complete, the most critical failure mode correctly identified, all single points of failure listed, ≥2 feasible suggestions for testing and mitigation strategies	FMEA ≥80% complete, the most critical failure mode correctly identified, max. 2 single points of failure missing, at least one feasible suggestion for testing and mitigation strategy	FMEA ≥50% complete, ≥1 critical failure mode identified, ≥1 single point of failure found, testing and mitigation strategy not feasible	FMEA < 50% complete, no critical failure mode identified, > 3 single points of failure missing, no suggestion for testing or mitigation strategy
Design Concept 25 %	Design is feasible & innovative, aligns with ConOps, all core functions are fully described	Design is feasible, aligns 100% with ConOps, and core functions are mostly described.	Design is mostly feasible, ≥ 50% alignment with ConOps, some core functions may be missing or unclear in description	Design is poorly feasible, misaligns with ConOps, many core functions unclear or missing
Presentation & Delivery 20 %	Clear, engaging delivery, logical structure, visuals well-integrated, no pacing issues	Mostly clear & engaging, visuals used well, ≤ 2 pacing/structure issues	Some clarity issues, visuals underused, inconsistent engagement	Disorganized, unclear, poor use of visuals, disengaging

10 Coding Help

The VEX Robot brain can be programmed in three different ways:

1. Python
2. C++
3. Blocks

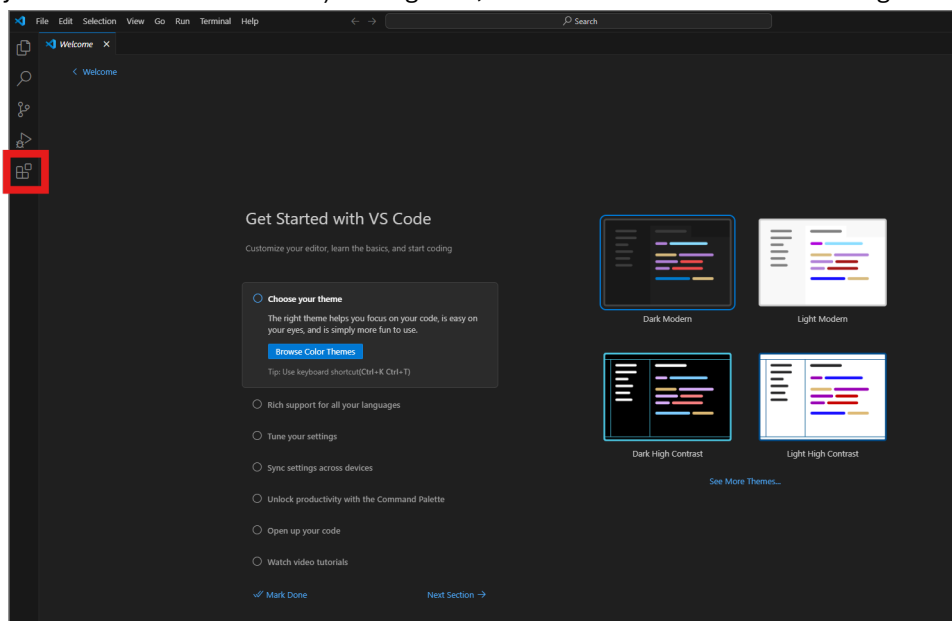
It is possible to use their special software (VEX Code IQ, [LINK](#)) to program the robot using either of the above methods. It's very user-friendly, but also designed for 11-year-old kids.

VEX Robotics has an extension for Visual Studio Code (VS Code, [LINK](#)), a professional IDE. We recommend using VS Code. It enables coding with both Python and C++, and provides more advanced code editing tools (such as refactoring, Git, etc.) compared to VEX Code IQ.

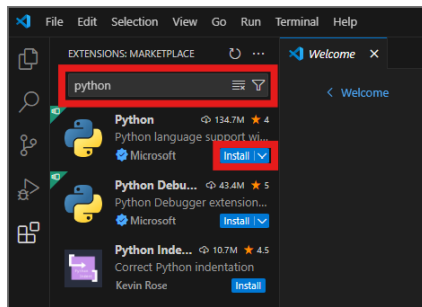
10.1 Installing the VEX VS Code Extension

As a first step, install Visual Studio Code (VS Code, [LINK](#)) on your computer. Then, follow the steps listed below.

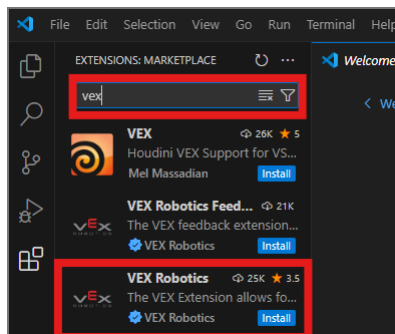
1. Launch VS Code
2. You will be greeted with the “Get Started with VS Code screen” (or just the welcome screen, if you’ve used VS Code before). Disregard it, and click on the “Extensions” Badge



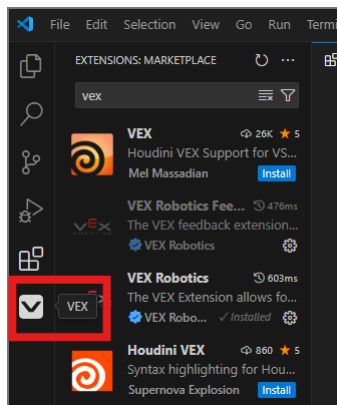
3. Install Python by searching for “Python” in the Marketplace and installing the first search result.



4. Install the VEX VS Code Extension by searching for “VEX” and installing the “VEX Robotics” extension

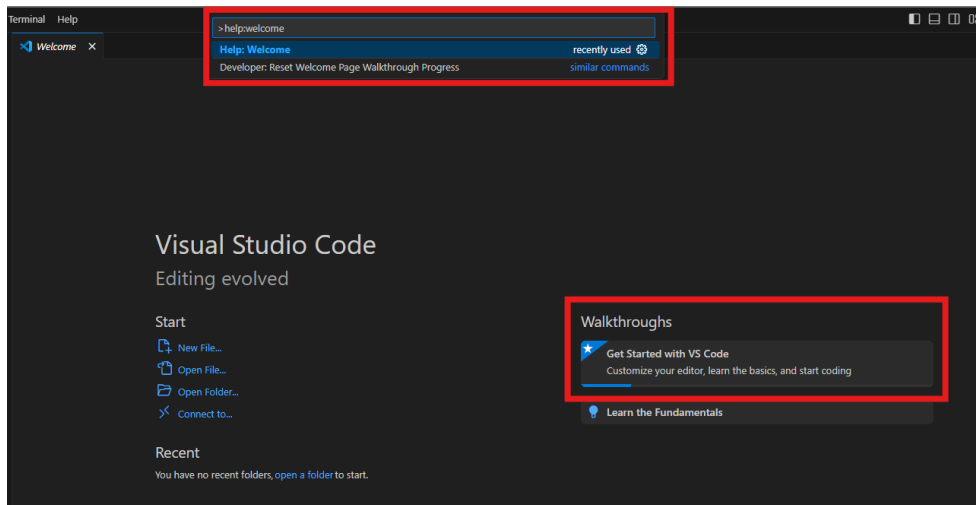


5. If the extension installed correctly, you will see a “VEX” Badge appear in the left-hand bar



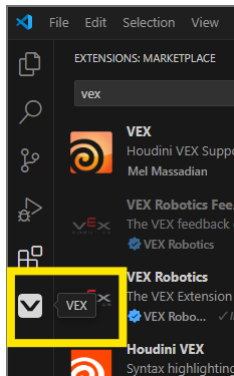
6. You're done and ready to continue
7. (Optional) If you'd like to return to the “Get Started with VS Code” screen, type “>Help: Welcome” into the top search bar (to get to the welcome screen) and click on the search result. On

the welcome screen, select the walkthrough to continue with “Get Started with VS Code”.

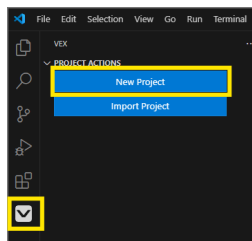


Using VEX VS Code

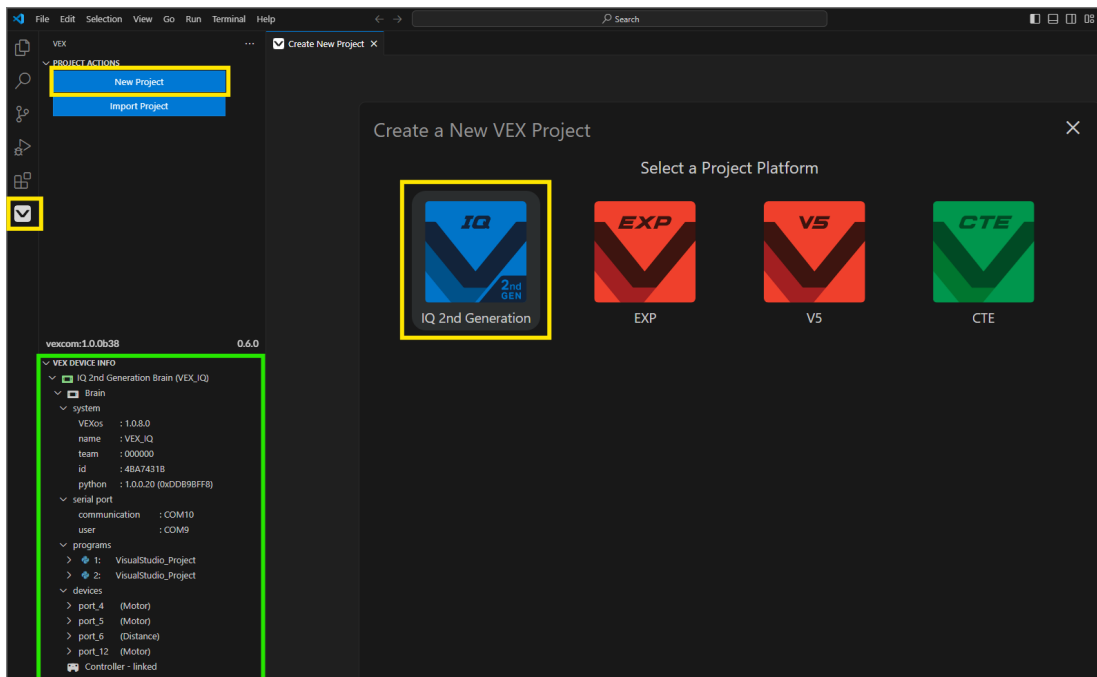
1. Power up the Robot Brain and connect it to your PC with the USB-C cable
2. In VS Code, click on the “VEX” Badge. You will see the properties of the Robot Brain and all connected devices.



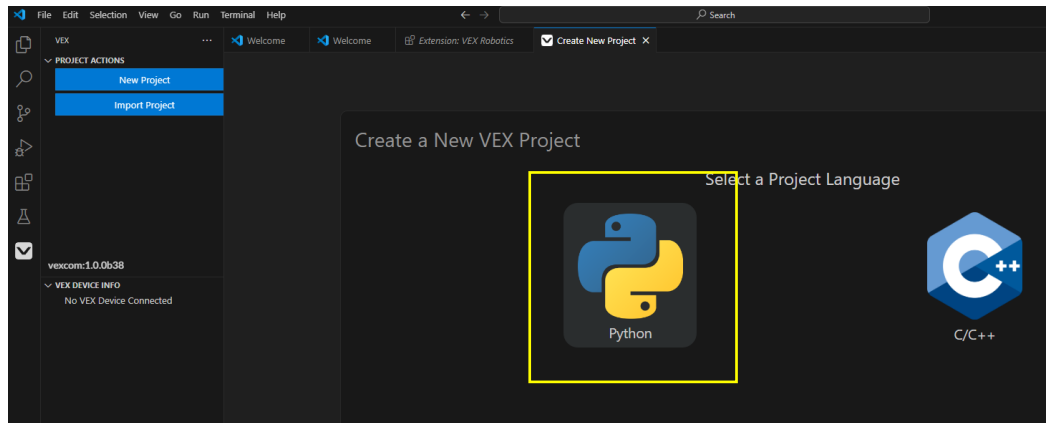
3. Click “New Project”



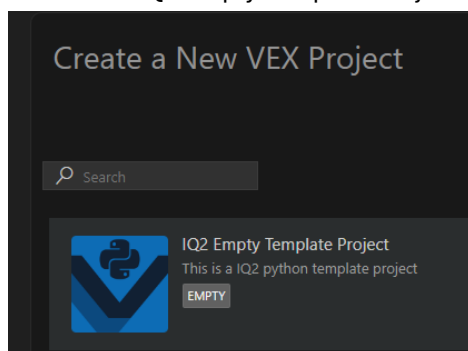
4. Click “IQ 2nd Generation”



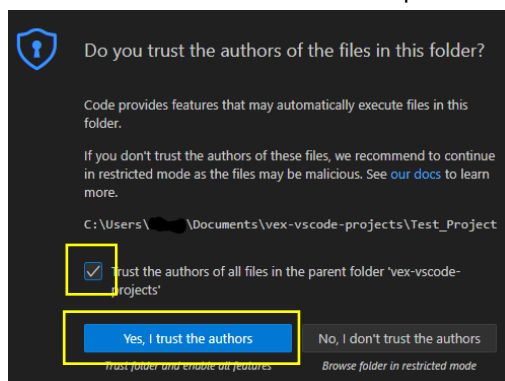
- Choose your programming language of choice (Python or C++). We recommend Python



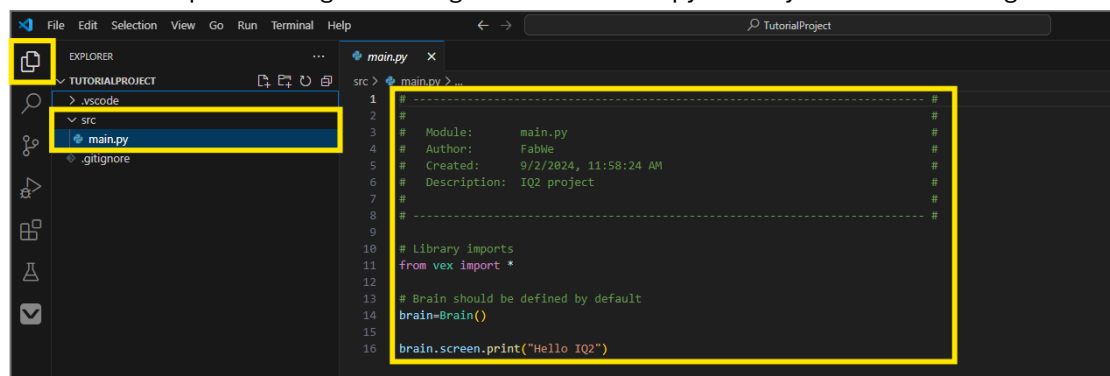
- Create an “IQ2 Empty Template Project” (or look at the provided templates)



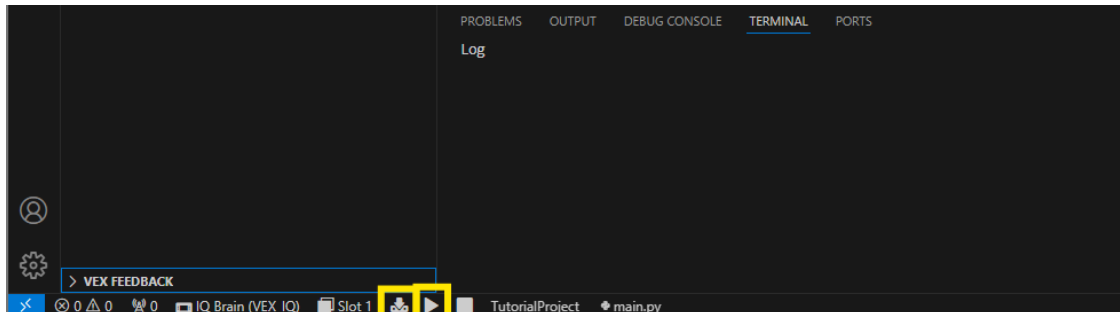
- Give your project a name and specify its storage location. Click “Create”. VS Code will generate a folder at the given location (with the name you entered in “Project Name”) and a corresponding folder structure.
- Trust the authors of the files in the parent folder



- Click on the “Explorer” Badge and navigate to “src > main.py”. Here you can start coding



10. To **Run your Code**, do **NOT** press the “Run Python File” on the top right of your screen. This will attempt to run the code locally, where the “vex” library is not available.
11. Instead, on the bottom of your screen, first press the “Build and Download” Button (which downloads the program you wrote on to the Robot Brain). Then press the “Play User Program” button down there. The download button only appears if a Brain is connected and recognised.



10.2 Important Hints & Links

10.2.1 Documentation

A full documentation of all functions is available here: [VEX Documentation](#)

The UI elements of the VEX VS Code extension are explained here: [UI Explainer](#)

10.2.2 Numpy, Matplotlib, etc.

The Python code runs on the Robot Brain, which only has base Python installed. Thus, any additional libraries are unavailable. Installing them from an SD Card might be possible, but this is beyond this course's scope.

10.2.3 Getting Data to the Computer

There are two different ways of transferring sensor data from the Robot Brain to the computer:

1. Print values to the console and Copy-Paste into excel file. (This is an officially supported method)
2. Buffer data to a mini-SD Card. Refer to this article: [Data Logging](#)

Note that the amount of memory available on the Robot Brain is quite limited. Thus, using method 1 might result in a memory overflow error.

10.2.4 General Hints

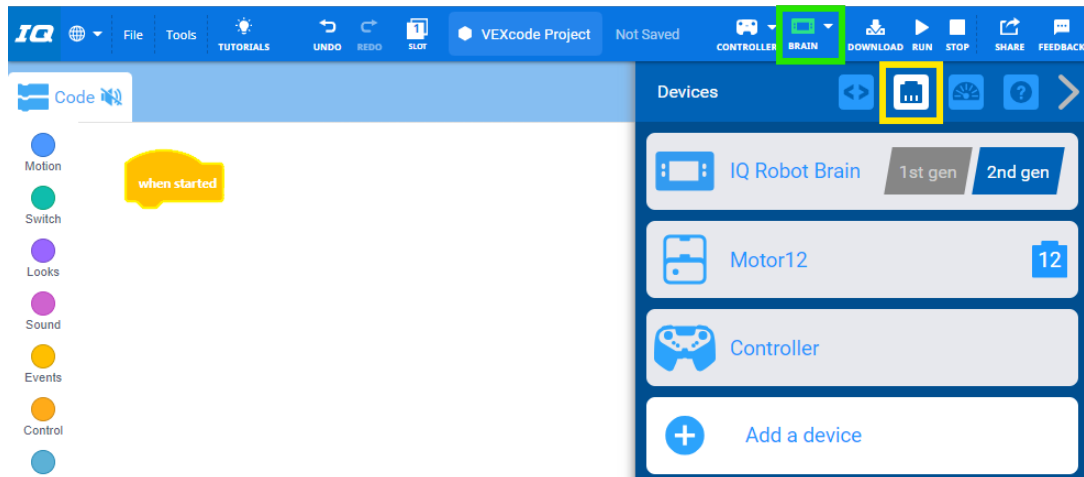
By default, when you command a motor to spin, the software immediately continues executing the next lines of code without waiting for the motor to stop. While this allows for simultaneous motions, it also means that if your program crashes, the motor may keep spinning indefinitely until you manually stop the program. Always keep an eye on your robot!

To use the controller, you need to turn on both the Brain and the Controller. The Brain has recognised the controller if its LED Light starts blinking. It is not possible to pair the Controller if a program is already being executed on the Brain.

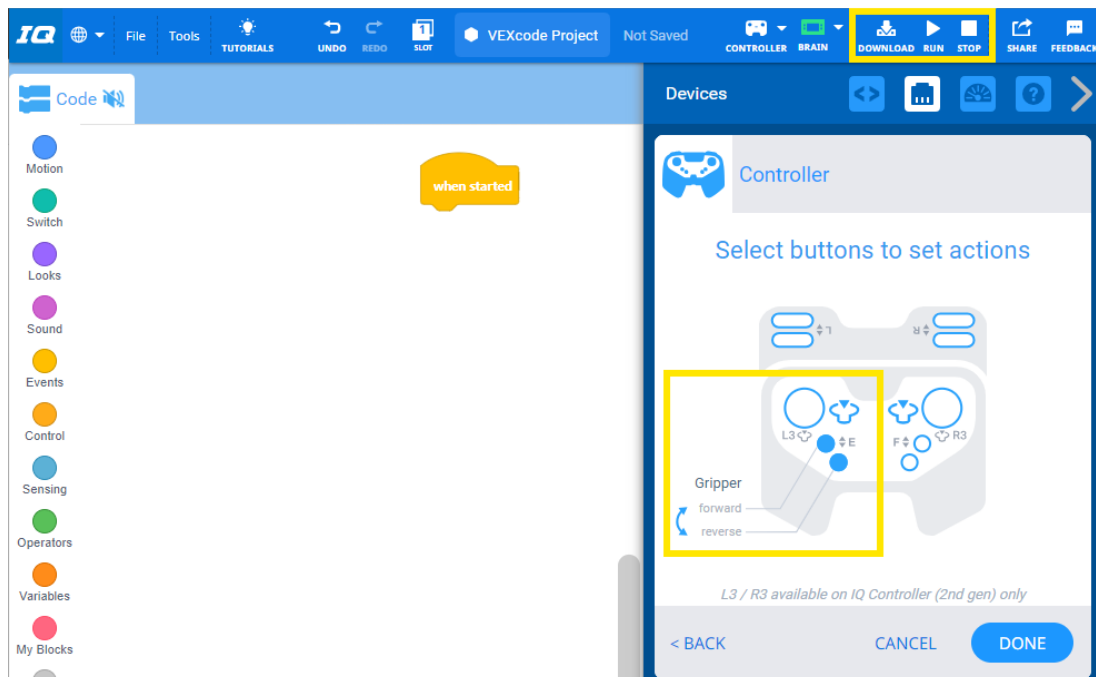
10.2.5 Manual Control

It might be beneficial to manually control your robot for testing using the controller. Assigning controller buttons to motor activations is most easily done in VEX Code IQ, [LINK](#). To do so, follow the given steps here:

1. Power up and connect your Robot Brain to your PC. Open VEX Code IQ. The “Brain” button is green if VEX Code recognises the connected Robot Brain.



2. Press the LAN-Port symbol and press “Add a Device”. Add a single “Motor” (2nd row on the left). Vex Code will ask you to select a connected Port, and you will be able to name the motor and reverse its directions. This adds a new motor to your devices
3. Click “Add a Device” again, and add a controller. By clicking on one of the buttons, you can loop through which button is assigned to which action.



4. Save your project and press “Download”. You can either press “Run” in VEX Code IQ or navigate to “Programs > /your Program name/ > Run” on the Robot Brain to run the program. Running the program in this context means that the Robot Brain is accepting inputs from the controller.

10.3 Testing Sensors and Motors

The functionality of the received sensors and motors can be checked using the testing script provided in *Utilities* → *Testing_snippet.zip*. If you haven't already, create a new VEX project as described in Section 10.1 and replace the *main.py* file under *src* -> *main.py* with the *main.py* file contained in the *Testing_snippet.zip* folder.

Physically connect all sensors and motors to their respective ports as defined in the script. Then run the code (see also Section 10.1) and follow the instructions displayed in the terminal. Check that each sensor and motor is working as expected.

Please report any malfunctioning component to the TAs to get a replacement.

```
# PORT CONFIG
SENSOR_TOUCH_PORT = Ports.PORT1
SENSOR_OPTICAL_PORT = Ports.PORT2
SENSOR_DISTANCE_PORT = Ports.PORT3
BUMPER_BUMPER_PORT = Ports.PORT4
MOTOR_1_PORT = Ports.PORT7
MOTOR_2_PORT = Ports.PORT8
MOTOR_3_PORT = Ports.PORT9
MOTOR_4_PORT = Ports.PORT10
```